Dyer
Innovation

CLAUDE CERTIFIED ARCHITECT · DOMAIN 4

Structured Output Mastery

A study guide for JSON Schema design, tool_use vs tool_choice, validation-retry loops, and the Message Batches API.

LAST UPDATED

April 27, 2026

THIS GUIDE EMPHASIZES

JSON Schema design (required/optional/nullable/enum) · tool_use vs tool_choice · Validation-retry loops · Message Batches API

Contents

01 Why Structured Output Matters in Production 3

02 Explicit Criteria vs Vague Instructions 4

03 tool_use for JSON Schemas — The Most Reliable Structured Output 5

04 tool_choice for Guaranteed Structured Output 6

05 Schema Design: Required, Optional, Nullable, Enum + "other" 7

- 5.1 Required vs optional 7
- 5.2 Nullable vs absent 7
- 5.3 Enum + "other" + detail string 7

06 Syntax Errors vs Semantic Errors — What Tool Use Solves and What It Doesn't 9

07 Validation-Retry Loops: When They Work and When They Don't 10

08 The detected_pattern Field for False Positive Analysis 11

09 The Message Batches API: 50% Savings, 24-Hour Window, Limitations 12

- 9.1 Failure handling with custom_id 12

10 Multi-Instance Review (Why 2 Claude Instances Beat Self-Review) 13

01 Why Structured Output Matters in Production

Free-form text from a language model is useful for humans and almost useless for code. The moment your application needs to do anything with Claude's response — store it, route it, validate it, sum it, hand it to another service — you need a guarantee about its shape.

The classic production failure mode:

- Model returned `customerName` on Monday
- `customer_name` on Tuesday
- `{"customer": {"name": "..."}}` on Wednesday
- Your ETL job has been silently dropping records ever since

Structured output is the discipline of removing that ambiguity. It is not one technique — it is a **stack of decisions**:

- The precision of your prompt
- The contract you express through a tool schema
- The way you force the model to honor that contract
- The validation layer that catches the cases where it didn't
- The review architecture that catches the cases validation can't see

Every layer in that stack is a separate exam topic, and the exam tests whether you can pick the right layer for a given failure.

Unifying mental model: Claude is a probabilistic component. Your job as the architect is to wrap that component in deterministic guardrails so that the system as a whole behaves predictably even when individual responses don't. The rest of this guide walks the stack from the prompt up.

02 Explicit Criteria vs Vague Instructions

The single highest-leverage change you can make to most prompts is replacing vague qualitative language with explicit, behaviorally-defined criteria. Vague instructions feel like they should work because they read like English directions you'd give a human — but the model has no shared context for what your team means by "be careful" or "only flag important things."

TRAP

Trap: the conservatism instruction.

- Telling Claude to "be conservative" or "only report high-confidence findings" does not improve precision in any measurable way
- It either has no effect, OR
- It suppresses true positives along with false ones, leaving you worse off

Replace each vague phrase with a behavioral rule that names the input pattern, the output, and the boundary case:

```
# Bad — vague, untestable
prompt = "Review this code carefully and only flag important issues."

# Good — explicit criteria with named report/skip categories
prompt = """
Review the diff. Flag a finding only when:
  - The comment claims behavior that contradicts what the code actually does
  - A null dereference is reachable from a public entrypoint
  - A SQL string is built by concatenation with user input

Do NOT flag:
  - Style preferences (spacing, naming) unless they violate the linter config
  - Local patterns that match the rest of the file
  - Performance concerns under 10ms in non-hot paths

Severity levels (with concrete examples in the few-shot block below):
  - critical: exploitable security issue, data corruption
  - major:    crash on common input, contract violation
```

```
  - minor:    confusing API, missing error case on rare input
"""
```

When false positives spike on a specific category:

- Temporarily disable that category while you fix the criteria
- Leaving a noisy detector on while you tune it erodes developer trust faster than the eventual fix can rebuild it

Why explicit criteria are foundational for the whole stack:

- Few-shot examples need a vocabulary to label
- Schemas need a categorical space to encode
- Review prompts need a target to evaluate against
- If you cannot write down the criteria, you do not yet have a problem statement worth shipping

---

03 `tool_use` for JSON Schemas — The Most Reliable Structured Output

The most reliable way to get structured output from Claude is **not** to ask for JSON in the prompt. It is to declare a tool with a JSON schema and let the model "call" that tool. The schema becomes the contract; the model's job is to fill it in.

Why this works: the API treats tool input as a typed object the model has to construct, not as free-form generation that happens to look like JSON.

Failure modes eliminated by construction:

- `{"name": "Acme"` with a missing brace
- JSON wrapped in markdown fences
- Preambles like "Sure, here's the JSON:"

A minimal extraction tool looks like this:

```
import anthropic

client = anthropic.Anthropic()

tools = [{
    "name": "extract_invoice",
    "description": (
        "Extract structured invoice data from the document. "
        "Use null for fields that are not present in the source — do not guess."
    ),
    "input_schema": {
        "type": "object",
        "properties": {
            "invoice_number": {"type": "string"},
            "issue_date":     {"type": "string", "format": "date"},
            "total_amount":   {"type": "number"},
            "currency":       {"type": "string", "enum": ["USD", "EUR", "GBP",
"other"]},
            "tax_id":         {"type": ["string", "null"]},
            "line_items": {
                "type": "array",
                "items": {
                    "type": "object",
                    "properties": {
                        "description": {"type": "string"},
                        "amount":      {"type": "number"}
                    },
```

```
                    "required": ["description", "amount"]
                }
            }
        },
        "required": ["invoice_number", "issue_date", "total_amount", "currency",
"line_items"]
    }
}]

response = client.messages.create(
    model="claude-sonnet-4-7",
    max_tokens=1024,
    tools=tools,
    tool_choice={"type": "tool", "name": "extract_invoice"},
    messages=[{"role": "user", "content": INVOICE_TEXT}],
)

# The structured output lives in the tool_use block, not in text.
extraction = next(b.input for b in response.content if b.type == "tool_use")
```

The schema does several jobs at once:

- Tells the model which fields to look for
- Which are required
- Which may be null
- Which belong to a closed set of values

Each of those signals is enforced before the response leaves the API. The model can still produce semantically wrong values — that's a separate problem covered later — but it cannot produce structurally wrong ones.

Bottom line:

- Compared to "please return JSON in the following format" prompting, schema-conformance failure rate drops by roughly an order of magnitude in practice
- There is no production reason to use prompt-only JSON when tool use is available

---

04 `tool_choice` for Guaranteed Structured Output

A tool definition by itself is not a guarantee. The model can still respond with plain text and ignore your tool entirely. The `tool_choice` parameter is what closes that gap. There are three values, and choosing the wrong one is a frequent exam distractor.

TOOL_CHOICE	BEHAVIOR	USE WHEN
`"auto"`	Model decides whether to call a tool or return text	Conversational agents that sometimes need tools, sometimes not
`"any"`	Model MUST call one of the available tools, but picks which one	You have several extraction tools (one per document type) and want the model to choose
`{"type": "tool", "name": "X"}`	Model MUST call exactly that tool	You have a single extraction tool and want a guaranteed structured response every time

The decision tree is:

```
Do you need a structured response on every call?
├── No  → "auto"  (model may return text)
└── Yes → Do you have one tool or several?
          ├── One     → {"type": "tool", "name": "..."}  (forced)
          └── Several → "any"  (must call one, model picks)
```

TRAP

Two anti-patterns worth flagging:

— `"auto"` is **not** "almost always calls the tool." It is "the model may decide a text response is more appropriate." If your downstream consumer assumes a tool input dict, `"auto"` will eventually surprise you.

— Do **not** use forced tool selection (`{"type": "tool", ...}`) when the document type is unknown and you actually need the model to discriminate — that's what `"any"` is for.

Exam scenario to watch for:

- A question describes an extraction pipeline that "occasionally returns plain text" and asks how to fix it
- Answer: switch from `auto` to `any` (or forced)
- Distractors to reject: stricter prompt, retry loop — these don't address the root cause

---

05 Schema Design: Required, Optional, Nullable, Enum + `"other"`

Schema design is where most teams accidentally re-introduce the problems tool use was supposed to solve.

Four dimensions to get right:

- Required vs optional
- Optional vs nullable
- Closed enum vs open enum
- How you handle the long tail

Required vs optional

- **Required** — the document is malformed without it (every invoice has a total)
- **Optional** (omitted from `required` list) — the field's absence is meaningful and you'd rather see it missing than fabricated

Nullable vs absent

- **Optional** — "the model may omit the key"
- **Nullable** — "the key must be present, but its value may be `null`"
- Why it matters: nullable forces every consumer to handle the null case explicitly; optional lets consumers `.get()` and move on

Exam-favored pattern:

- Declare a field as `["string", "null"]` when the source document might genuinely lack it
- Pair with the instruction "use null instead of guessing"
- This is the single most effective anti-fabrication pattern in the schema toolkit

```
{
  "tax_id":           {"type": ["string", "null"]},
  "purchase_order":   {"type": ["string", "null"]},
```

```
  "shipping_address": {"type": ["object", "null"], "properties": { ... }}
}
```

Enum + `"other"` + detail string

Closed enums are clean until the day a real document doesn't fit any of your categories. Then the model either picks the closest-but-wrong value, or refuses to extract, or starts hallucinating a category outside the enum. The escape valve is the `"other"` pattern:

```
{
  "category": {
    "type": "string",
    "enum": ["billing", "technical", "account", "shipping", "other"]
  },
  "category_detail": {
    "type": ["string", "null"],
    "description": "Required when category is 'other'. A short phrase describing the
actual category."
  }
}
```

Why this works:

- The model has a graceful degradation path
- Downstream, monitor the rate of `"other"` responses to discover categories you missed
- For ambiguous inputs, add an `"unclear"` enum value — signals the human reviewer that the model recognized the ambiguity rather than guessing through it

TRAP

Trap to avoid:

— A closed enum with no escape valve in a domain you don't fully understand

— Add `"other"` from day one — it costs nothing and prevents a class of failures that's hard to debug after the fact

06 Syntax Errors vs Semantic Errors — What Tool Use Solves and What It Doesn't

Tool use eliminates syntax errors:

- Malformed JSON
- Missing required fields
- Wrong types
- Values outside the enum

These are gone the moment you wire up the schema. Treat them as a solved problem.

Tool use does NOT eliminate semantic errors:

- Line items that don't sum to the stated total
- A date in the right format but the wrong year
- An address parsed into the wrong fields
- A fabricated tax ID for a document that didn't have one

The schema is satisfied; the values are wrong. This is the most consistently tested distinction in the structured-output portion of the exam, and it appears in two flavors:

PROBLEM	CATEGORY	WHAT ACTUALLY FIXES IT
`total_amount` field comes back as a string	Syntax	`tool_use` with proper JSON schema
`total_amount` is the number, but it doesn't equal `sum(line_items)`	Semantic	Self-check field + validation-retry, or downstream reconciliation
Extraction sometimes returns prose instead of structured data	Syntax	`tool_choice: "any"` or forced
Model invents a tax_id for an invoice that didn't list one	Semantic	Nullable field + explicit "use null" instruction

The self-correction pattern that helps with semantic errors is to add **derived fields to the schema** that the model has to compute and that you then cross-check:

```
"properties": {
    "stated_total":     {"type": "number"},
    "calculated_total": {"type": "number",
                         "description": "Sum of line_items[].amount"},
    "totals_match":     {"type": "boolean"},
    "conflict_detected":{"type": "boolean",
                         "description": "True if the document contradicts itself"}
}
```

When `totals_match` is `false`:

- You have a semantic error the schema couldn't have caught
- Route those rows to human review
- The schema didn't fix the error — but it surfaced it instead of letting it through silently

---

07 Validation-Retry Loops: When They Work and When They Don't

A validation-retry loop catches the cases where validation fails and re-prompts the model with the specific error. It is the second line of defense after the schema, and like all retry mechanisms, it works for some failure classes and is useless for others.

The shape of the loop:

```
from pydantic import ValidationError

def extract_with_retry(document: str, max_retries: int = 2):
    messages = [{"role": "user", "content": document}]
    for attempt in range(max_retries + 1):
        response = client.messages.create(
            model="claude-sonnet-4-7",
            max_tokens=1024,
            tools=tools,
            tool_choice={"type": "tool", "name": "extract_invoice"},
            messages=messages,
        )
        block = next(b for b in response.content if b.type == "tool_use")
        try:
            return Invoice.model_validate(block.input)        # success
        except ValidationError as e:
            messages.append({"role": "assistant", "content": response.content})
            messages.append({"role": "user", "content": (
                f"The extraction failed validation with these errors:\n{e}\n\n"
                "Re-extract using the same tool, correcting only the listed errors.
"
                "If a field is genuinely absent from the source, return null."
            )})
    raise RuntimeError("Extraction did not validate after retries.")
```

RULE

Decision rule (load-bearing):

— Retries help when the information is in the source document and the model formatted it wrong

— Retries do NOT help when the information is absent

FAILURE	INFORMATION PRESENT?	RETRY HELPS?
Date returned as `"03/14"` instead of `"2026-03-14"`	Yes	Yes
Amount returned as a string instead of a number	Yes	Yes
Extra field outside the schema	Yes	Yes
`tax_id` was never in the document; model invented one	No	No — the retry will invent a different one
Document is a scan with the relevant region cut off	No	No

For the second class (information absent), the right answer:

- Allow `null` in the schema
- Instruct the model to use it
- Treat persistent retry failures as a signal to route the document to human review
- Looping more times against missing information just burns tokens

TIP

Exam distractor:

"increase the retry count" for information-absent failures. Reject it.

08 The `detected_pattern` Field for False Positive Analysis

Once your extraction pipeline is running at scale, you'll have a steady drip of false positives — categorizations that came back wrong, fields that misfired on a specific document layout, edge cases nobody anticipated. The `detected_pattern` field is a schema-level mechanism for letting the model annotate **what triggered each finding**, so downstream reviewers can cluster errors by pattern instead of triaging them one at a time.

```
{
  "findings": {
    "type": "array",
    "items": {
      "type": "object",
      "properties": {
        "category":         {"type": "string", "enum": [...]},
        "evidence_excerpt": {"type": "string"},
        "detected_pattern": {
          "type": "string",
          "description": (
            "A short label for the surface pattern that triggered this finding. "
            "Examples: 'inline_total_after_subtotal',
'currency_symbol_in_amount_field', "
            "'multi_page_table_continuation'. Use the same label across documents "
            "when the same pattern recurs."
          )
        },
        "confidence": {"type": "string", "enum": ["high", "medium", "low"]}
      },
      "required": ["category", "evidence_excerpt", "detected_pattern"]
    }
  }
}
```

Why this is high-leverage:

- Analyze a week of false positives and see that 60% carry `detected_pattern: "currency_symbol_in_amount_field"` → you have a fix target
- Update the prompt or schema description to handle currency symbols explicitly
- Without `detected_pattern`, you'd be staring at sixty unrelated-looking errors

Why it makes calibrated human review feasible:

- Confidence scores alone are unreliable as escalation signals — they're famously poorly calibrated
- Confidence paired with a pattern label gives reviewers a reason to trust or distrust each finding
- And gives engineers a way to measure improvement over time

---

09 The Message Batches API: 50% Savings, 24-Hour Window, Limitations

The Message Batches API runs your requests asynchronously, returns results within 24 hours, and costs roughly 50% less than synchronous calls. The savings are real. The limitations are also real, and the exam tests whether you can match the API choice to the latency requirement.

What you get:

- ~50% cost reduction vs synchronous
- Up to 100,000 requests per batch
- A `custom_id` you supply on submission and that comes back on every result, so you can correlate request → response

What you give up:

- **No latency SLA.** Most batches return well under 24 hours; some take the full window.
- **No multi-turn tool calling within a single request.** Each batch entry is one shot.
- No streaming.

The decision matrix:

WORKLOAD	API
Pre-merge CI check that blocks the PR	Synchronous
Interactive customer support reply	Synchronous
Overnight code review across the whole repo	Batch
Weekly compliance audit of last week's transcripts	Batch
Nightly test-case generation from yesterday's commits	Batch
Backfilling extractions across a 100k-document archive	Batch

The math you need to be ready to do:

- SLA = 30 hours, batch up to 24 hours → submit every 4 hours and still meet the SLA with a buffer
- SLA = 4 hours → you cannot use the Batch API at all (even a fast batch can miss that window)

Failure handling with `custom_id`

Batch responses are independent — one bad request doesn't fail the batch. The recovery pattern is:

```
results = client.messages.batches.results(batch_id)

failed = []
for line in results:
    if line.result.type == "errored":
        # Look up the original request by custom_id and queue for resubmission
        original = original_requests_by_id[line.custom_id]
        failed.append(transform_for_retry(original))   # e.g., chunk if too long

if failed:
    client.messages.batches.create(requests=failed)
```

`custom_id` rules:

- It's the join key between your input batch and the output stream
- Without it, you cannot tell which response goes with which input
- Non-negotiable for any batch workload — generate a deterministic ID per input and persist it

10 Multi-Instance Review (Why 2 Claude Instances Beat Self-Review)

Why self-review under-performs:

- The model that just generated the response carries the reasoning context that produced it
- That context biases it toward agreeing with the choice it already made
- Self-review catches some surface errors but consistently misses deeper "the whole approach is wrong" failures

The fix is structural, not prompt-based:

- Spin up a second Claude instance with no exposure to the generation reasoning
- Give it only the artifact and a review rubric
- Let it critique fresh

```
def two_instance_review(diff: str) → dict:
    # Instance 1: generate findings
    generation = client.messages.create(
        model="claude-sonnet-4-7",
        max_tokens=2048,
        messages=[{"role": "user", "content": GENERATION_PROMPT.format(diff=diff)}],
    )
    findings = extract_findings(generation)

    # Instance 2: independent review — gets ONLY the diff and the findings,
    # not the generation transcript.
    review = client.messages.create(
        model="claude-sonnet-4-7",
        max_tokens=1024,
        messages=[{"role": "user", "content": REVIEW_PROMPT.format(
            diff=diff,
            findings=findings,
        )}],
    )
    return merge(findings, parse_review(review))
```

TIP

Two design notes:

— The review instance must NOT see the generation conversation — that's the whole point. If you accidentally pass the generation transcript into the review prompt, you've reproduced self-review with extra steps.

— Extended thinking on the original instance is not a substitute. More thinking from the same instance still has the same context bias. The independence is what's load-bearing.

Exam phrasing to recognize:

- "Inconsistent code review," "attention dilution," or "the model misses obvious bugs in its own generated code" → multi-instance review
- Distractors to reject: "increase the temperature," "use extended thinking," "add more few-shot examples"
- The correct answer is the structural separation

---

11 Multi-Pass Review: Per-File + Cross-File Integration

The single-pass failure mode on a 14-file PR:

- Inconsistent findings
- Missed bugs in middle files
- Contradictory observations across files

Cause: attention dilution. The model can hold a few hundred lines of code in sharp focus, but at multi-thousand-line scope it starts skimming. Throwing more context window at the problem doesn't fix attention quality.

The pattern that does fix it is a **two-pass structure**:

1. **Per-file pass — narrow scope, deep focus.** Each file gets its own review call with its own context. Findings are local: dead code, off-by-one bugs, missing error handling, contract violations within the file.
2. **Cross-file integration pass — broad scope, narrow purpose.** A single follow-up call that gets summaries (not full text) of every file plus the diffs between them. Findings are integration-level: a function whose new signature breaks a caller in another file, an enum value added in one place but not handled in the consumer, a migration that's missing a corresponding model change.

```
def multi_pass_review(pr_files: list[File]) → Review:
    per_file = []
    for f in pr_files:
        per_file.append(review_one_file(f))   # narrow + deep, one call each

    cross_file = review_integration(
        file_summaries=[summarize(f, findings) for f, findings in zip(pr_files,
per_file)],
        cross_file_diffs=compute_call_graph_changes(pr_files),
    )
    return Review(per_file=per_file, cross_file=cross_file)
```

TIP

Two distractors the exam likes for this scenario:

- "Use a model with a larger context window" — context size is not the constraint, attention quality is
- "Require consensus across multiple runs before reporting a finding" — consensus suppresses intermittent-but-real bugs along with the noise

Per-file plus cross-file is the right answer because it matches the structure of where bugs actually live.

---

12 Quick Reference

When to use which structured-output mechanism

MECHANISM	USE WHEN	AVOID WHEN
Plain prompt asking for JSON	Never (in production)	Always
`tool_use` with schema, `tool_choice: "auto"`	Conversational agent that sometimes uses tools	You need a structured response on every call
`tool_use` with schema, `tool_choice: "any"`	Multiple extraction tools, one per document type	Single extraction shape (force the specific tool)
`tool_use` with schema, forced (`{"type": "tool", "name": "X"}`)	Single extraction shape, every call	The model needs to choose between several tools
Prefilling assistant turn with `{`	Legacy patterns where tool_use isn't available	You can use tool_use (which is almost always)

`tool_choice` decision tree

```
Will the model always need to return structured data?
├── No  → "auto"
└── Yes → Is there only one tool you want it to call?
          ├── Yes → {"type": "tool", "name": "..."}  (forced)
          └── No  → "any"
```

Validation-retry decision tree

```
Did validation fail?
├── No  → return result
└── Yes → Is the missing/wrong information PRESENT in the source document?
          ├── Yes → Retry with the specific validation error in the message
          │         (cap at 2-3 attempts)
```

```
└── No  → Allow null in the schema, instruct the model to use it,
          and route persistent failures to human review.
          Do NOT keep retrying.
```

Schema design checklist

- Required fields are the ones whose absence makes the document invalid
- Optional fields (omitted from `required`) are the ones the source may legitimately lack
- Nullable (`["string", "null"]`) forces a present-but-null value — use to suppress fabrication
- Every closed enum gets an `"other"` value plus a `_detail` string for the long tail
- For ambiguity, add an `"unclear"` enum value rather than forcing a guess
- For semantic self-checks, add derived fields (`calculated_total`, `totals_match`, `conflict_detected`)
- For false-positive analysis, add `detected_pattern` so reviewers can cluster errors

Sync vs Batch API

PROPERTY	SYNCHRONOUS	BATCH
Cost	Baseline	~50% off
Latency	Seconds	Up to 24 hours, no SLA
Multi-turn tool calling	Yes	No
Use for	Blocking checks, interactive responses	Overnight reports, audits, backfills

Review architecture

- **Self-review** (same instance evaluates own output) → biased by generation context, weak
- **Two-instance review** (independent reviewer Claude, no generation transcript) → catches what self-review misses
- **Single-pass review of large PRs** → attention dilution, inconsistent findings
- **Per-file + cross-file integration pass** → matches the structure of where bugs live

Things tool use does and does not solve

SOLVED BY TOOL USE	NOT SOLVED BY TOOL USE
Malformed JSON	Wrong values in the right fields
Missing required fields	Hallucinated content for absent information
Wrong types	Line items that don't sum to total
Values outside the enum	Plausible-but-incorrect categorization
Markdown fences around the JSON	Date in valid format but wrong year

When the failure is in the right column, the fix is somewhere else in the stack:

- Nullable fields
- Derived self-check fields
- Validation-retry
- Multi-instance review
- Human-in-the-loop routing

Match the layer to the failure and the system holds together.

A language model is a probabilistic component. Your job as the architect is to wrap it in deterministic guardrails so the system behaves predictably even when individual responses don't.